

OSSP (Operating Systems And Systems Programming) - 25CS2104E

1(a). Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program1a.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    char cmd[100];  
    char *args[10];  
    int i = 0;  
  
    printf("Enter command: ");  
    fgets(cmd, sizeof(cmd), stdin);  
    cmd[strcspn(cmd, "\n")] = '\0';  
  
    char *token = strtok(cmd, " ");  
    while (token != NULL)  
    {  
        args[i++] = token;  
        token = strtok(NULL, " ");  
    }  
    args[i] = NULL;  
  
    pid_t pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child PID = %d\n", getpid());  
        execvp(args[0], args);  
        perror("exec failed");  
        exit(1);  
    }  
    else  
    {  
        wait(NULL);  
        printf("Parent PID = %d\n", getpid());  
        printf("Child terminated\n");  
    }  
  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc program1a.c -o program1a
praks@praks:~$ ./program1a
Enter command: date
Child PID = 294
Tue Sep 29 04:02:54 UTC 2026
Parent PID = 293
Child terminated
praks@praks:~$ |
```

1(b). Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Commands and Output:

```
praks@praks:~$ uname -a
Linux vm 6.18.44-fc-v37 #1 SMP PREEMPT_DYNAMIC @0 x86_64 x86_64 x86_64 GNU/Linux
praks@praks:~$ lscpu | head -14
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                52 bits physical, 57 bits virtual
Byte Order:                  Little Endian
CPU(s):                       1
On-line CPU(s) list:         0
Vendor ID:                   GenuineIntel
Model name:                  Intel(R) Xeon(R) Processor @ 2.10GHz
CPU family:                  6
Model:                       207
Thread(s) per core:          1
Core(s) per socket:          1
Socket(s):                   1
Stepping:                    2
praks@praks:~$ |
```

```

praks@praks:~$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
zram0 253:0    0   0B  0 disk 
vda   254:0    0 256G  0 disk /
vdb   254:16   0  9.6M  1 disk /opt/rcclone
vdc   254:32   0  676K  1 disk /mnt/skills/public
vdd   254:48   0  5.7M  1 disk /mnt/skills/examples
praks@praks:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           3.9Gi         260Mi        3.7Gi         12Mi        140Mi        3.7Gi
Swap:           0B           0B           0B
praks@praks:~$ ps
  PID TTY          TIME CMD
  287 pts/0    00:00:00 bash
  300 pts/0    00:00:00 ps
praks@praks:~$ top -bn1 | head -12
top - 04:02:55 up 1 min,  0 user,  load average: 0.11, 0.03, 0.01
Tasks:  52 total,   1 running,  51 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  4000.3 total,  3788.5 free,   260.9 used,   140.0 buff/cache
MiB Swap:   0.0 total,    0.0 free,    0.0 used.  3739.4 avail Mem

   PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0   17452   4568  4200  S   0.0   0.1   0:00.64 process_a+
    2 root        20   0        0        0        0  S   0.0   0.0   0:00.00 kthreadd
    3 root        20   0        0        0        0  S   0.0   0.0   0:00.00 pool_work+
    4 root         0 -20        0        0        0  I   0.0   0.0   0:00.00 kworker/R+
    5 root         0 -20        0        0        0  I   0.0   0.0   0:00.00 kworker/R+
praks@praks:~$ |

```

Explanation: CPU: lscpu shows the physical CPU, but the OS scheduler shares it between all processes (ps, top), so every process behaves as if it has its own CPU. Memory: free shows the physical RAM, which the OS turns into a private virtual address space for each process. Storage: lsblk shows the disks, which the OS presents as files and directories through the file system. I/O devices: keyboard, terminal and disks are used through device files and system calls like read() and write(), so programs never touch the hardware directly. uname shows the kernel that provides all these services.

2(a). Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program2a.c  
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
  
int main()  
{  
    int src, dest, n;  
    char buf[100];  
  
    src = open("sample.txt", O_RDONLY);  
    dest = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);  
  
    if (src < 0 || dest < 0)  
    {  
        perror("open");  
        return 1;  
    }  
  
    while ((n = read(src, buf, sizeof(buf))) > 0)  
        write(dest, buf, n);  
  
    close(src);  
    close(dest);  
  
    printf("File copied successfully\n");  
    return 0;  
}
```

Output:

```
praks@praks:~$ echo "Welcome to OSSP practical" > sample.txt  
praks@praks:~$ cat sample.txt  
Welcome to OSSP practical  
praks@praks:~$ gcc program2a.c -o program2a  
praks@praks:~$ ./program2a  
File copied successfully  
praks@praks:~$ cat copy.txt  
Welcome to OSSP practical  
praks@praks:~$ |
```

Explanation: Each call to `open()`, `read()`, `write()` or `close()` executes a trap instruction. The CPU switches from user mode to kernel mode, the kernel performs the work (file system and disk access), stores the result and returns to user mode, and the program continues. So every system call is one user space to kernel space and back transition.

2(b). Use the strace utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

Commands and Output:

```
praks@praks:~$ strace -o trace.txt cat sample.txt
Welcome to OSSP practical
praks@praks:~$ wc -l trace.txt
116 trace.txt
praks@praks:~$ tail -12 trace.txt
fstat(3, {st_mode=S_IFREG|0644, st_size=26, ...}) = 0
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f0c8c2ed000
read(3, "Welcome to OSSP practical\n", 131072) = 26
write(1, "Welcome to OSSP practical\n", 26) = 26
read(3, "", 131072) = 0
munmap(0x7f0c8c2ed000, 139264) = 0
close(3) = 0
close(1) = 0
close(2) = 0
exit_group(0) = ?
+++ exited with 0 +++
praks@praks:~$ |
```

```
praks@praks:~$ strace -c cat sample.txt
Welcome to OSSP practical
% time      seconds  usecs/call   calls   errors syscall
-----
40.50      0.000081      3         22      0      mmap
29.00      0.000058      1         31     14      openat
6.00       0.000012      0         18      0      fstat
5.00       0.000010      3          3      0      mprotect
4.50       0.000009      0         19      0      close
4.50       0.000009      4          2      0      munmap
3.50       0.000007      7          1      0      write
2.00       0.000004      1          3      0      read
1.50       0.000003      1          3      0      brk
0.50       0.000001      1          1      0      arch_prctl
0.50       0.000001      1          1      0      futex
0.50       0.000001      1          1      0      fadvise64
0.50       0.000001      1          1      0      set_robust_list
0.50       0.000001      1          1      0      prlimit64
0.50       0.000001      1          1      0      getrandom
0.50       0.000001      1          1      0      rseq
0.00       0.000000      0          2      0      pread64
0.00       0.000000      0          1      1      access
0.00       0.000000      0          1      0      execve
0.00       0.000000      0          1      0      set_tid_address
-----
100.00     0.000200      1        114     15 total
praks@praks:~$ |
```

Explanation: Sequence: `execve` (load program), `brk` and `mmap` (memory setup), `openat`, `read`, `close` (load libc), then `openat("sample.txt")`, `fstat`, `read` (get data from file), `write(1, ...)` (print on terminal), and finally `close` and `exit_group` (end process). Kernel services involved: process management (`execve`, `exit_group`), memory management (`brk`, `mmap`, `munmap`), file system (`openat`, `fstat`, `read`, `close`) and I/O (`write` to terminal).

3(a). Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program3a.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    printf("Before fork : PID = %d, PPID = %d\n", getpid(), getppid());  
  
    pid_t pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child : PID = %d, PPID = %d (Running)\n", getpid(), getppid());  
        sleep(2);  
        printf("Child : exiting (Terminated)\n");  
        exit(0);  
    }  
    else  
    {  
        printf("Parent : PID = %d, Child = %d (Waiting)\n", getpid(), pid);  
        wait(NULL);  
        printf("Parent : child finished (Running)\n");  
    }  
  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc program3a.c -o program3a  
praks@praks:~$ ./program3a  
Before fork : PID = 326, PPID = 287  
Parent : PID = 326, Child = 327 (Waiting)  
Child : PID = 327, PPID = 326 (Running)  
Child : exiting (Terminated)  
Parent : child finished (Running)  
praks@praks:~$ |
```

3(b). Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Commands and Output:

```
praks@praks:~$ yes > /dev/null &
[1] 328
praks@praks:~$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
  328 R    yes
praks@praks:~$ grep State /proc/$!/status
State: R (running)
praks@praks:~$ sleep 300 &
[2] 331
praks@praks:~$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
  331 S    sleep 300
praks@praks:~$ grep State /proc/$!/status
State: S (sleeping)
praks@praks:~$ |
```

```
praks@praks:~$ kill -STOP $!
praks@praks:~$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
  331 T    sleep 300

[2]+  Stopped                  sleep 300
praks@praks:~$ grep State /proc/$!/status
State: T (stopped)
praks@praks:~$ kill -CONT $!
praks@praks:~$ kill $!
praks@praks:~$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
[2]+  Terminated              sleep 300
praks@praks:~$ |
```

```
praks@praks:~$ top -bn1 | head -10
top - 04:03:03 up 1 min,  0 user,  load average: 0.17, 0.04, 0.01
Tasks:  53 total,   2 running,  51 sleeping,   0 stopped,   0 zombie
%Cpu(s): 22.2 us, 77.8 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  4000.3 total,  3791.7 free,   257.7 used,   140.2 buff/cache
MiB Swap:   0.0 total,   0.0 free,   0.0 used.  3742.6 avail Mem

   PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
   328 praks    20   0   3136   1900  1784 R   90.9   0.0   0:02.89  yes
     1 root       20   0  17452   4568  4200 S    0.0   0.1   0:00.65  process_a+
     2 root       20   0     0     0     0 S    0.0   0.0   0:00.00  kthreadd

praks@praks:~$ kill %1
praks@praks:~$ ps
  PID TTY          TIME CMD
  287 pts/0        00:00:00 bash
  339 pts/0        00:00:00 ps
[1]+  Terminated              yes > /dev/null
praks@praks:~$ |
```

Explanation: yes never waits, so it is always Running/Ready (R) and top shows about 100% CPU. sleep 300 waits for a timer, so it is Waiting (S, sleeping). After kill -STOP the state becomes T (stopped), and after kill the process is Terminated and no longer appears in ps. Linux uses the same state R for both Ready and Running.

4(a). Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program4a.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t p[3];  
    int i, status;  
  
    for (i = 0; i < 3; i++)  
    {  
        p[i] = fork();  
        if (p[i] == 0)  
        {  
            sleep(3 - i);  
            printf("Child %d (PID %d) finished\n", i + 1, getpid());  
            exit(i + 1);  
        }  
    }  
  
    printf("--- Using wait() : any child ---\n");  
    pid_t w = wait(&status);  
    printf("wait() returned PID %d, status %d\n", w, WEXITSTATUS(status));  
  
    printf("--- Using waitpid() : specific child ---\n");  
    waitpid(p[0], &status, 0);  
    printf("waitpid() for Child 1 (PID %d), status %d\n", p[0], WEXITSTATUS(status));  
    waitpid(p[1], &status, 0);  
    printf("waitpid() for Child 2 (PID %d), status %d\n", p[1], WEXITSTATUS(status));  
  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc program4a.c -o program4a  
praks@praks:~$ ./program4a  
--- Using wait() : any child ---  
Child 3 (PID 348) finished  
wait() returned PID 348, status 3  
--- Using waitpid() : specific child ---  
Child 2 (PID 347) finished  
Child 1 (PID 346) finished  
waitpid() for Child 1 (PID 346), status 1  
waitpid() for Child 2 (PID 347), status 2  
praks@praks:~$ |
```

Explanation: wait() returns for whichever child finishes first (Child 3 here), so the order cannot be chosen. waitpid() waits for one particular child, so Child 1 and Child 2 were collected in the order we wanted, even though Child 2 finished earlier.

4(b). Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 zombie.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
  
int main()  
{  
    pid_t pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child exiting, PID = %d\n", getpid());  
        exit(0);  
    }  
    else  
    {  
        printf("Parent sleeping, PID = %d\n", getpid());  
        sleep(10);  
    }  
  
    return 0;  
}
```

```
praks@praks: ~  
GNU nano 8.7.1 nozombie.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child exiting, PID = %d\n", getpid());  
        exit(0);  
    }  
    else  
    {  
        printf("Parent sleeping, PID = %d\n", getpid());  
        wait(NULL);  
        sleep(10);  
    }  
  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc zombie.c -o zombie
praks@praks:~$ ./zombie &
[1] 354
praks@praks:~$ Child exiting, PID = 355
Parent sleeping, PID = 354
praks@praks:~$ ps -o pid,ppid,stat,cmd | grep -E "PID|zombie"
  PID  PPID  STAT  CMD
  354   287    S    ./zombie
  355   354    Z    [zombie] <defunct>
  357   287  S+    grep -E PID|zombie
praks@praks:~$ |
```

```
praks@praks:~$ gcc nozombie.c -o nozombie
praks@praks:~$ ./nozombie &
[1] 364
praks@praks:~$ Parent sleeping, PID = 364
Child exiting, PID = 365
praks@praks:~$ ps -o pid,ppid,stat,cmd | grep -E "PID|nozombie"
  PID  PPID  STAT  CMD
  364   287    S    ./nozombie
  367   287  S+    grep -E PID|nozombie
praks@praks:~$ |
```

Explanation: In zombie.c the child exits at once but the parent never calls wait(), so the child stays in the process table with state Z (defunct). In nozombie.c the parent calls wait(), which collects the exit status and removes the child from the table, so no defunct process is seen.

5(a). Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program5a.c  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
#include <sys/time.h>  
  
int main()  
{  
    int fd[2], i, x, count = 0;  
    struct timeval t1, t2;  
  
    pipe(fd);  
  
    if (fork() == 0)  
    {  
        close(fd[1]);  
        while (read(fd[0], &x, sizeof(x)) > 0)  
        {  
            if (x <= 3)  
                printf("Consumer got %d\n", x);  
            count++;  
        }  
        printf("Consumer received %d integers\n", count);  
        close(fd[0]);  
    }  
    else  
    {  
        close(fd[0]);  
        gettimeofday(&t1, NULL);  
        for (i = 1; i <= 100000; i++)  
        {  
            if (i <= 3)  
                printf("Producer sent %d\n", i);  
            write(fd[1], &i, sizeof(i));  
        }  
        close(fd[1]);  
        wait(NULL);  
        gettimeofday(&t2, NULL);  
  
        double t = (t2.tv_sec - t1.tv_sec) + (t2.tv_usec - t1.tv_usec) / 1e6;  
        printf("Time taken = %f sec\n", t);  
        printf("Data sent = %d bytes\n", (int)(100000 * sizeof(int)));  
        printf("Speed = %.2f MB/s\n", (100000 * sizeof(int)) / t / 1e6);  
    }  
  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc program5a.c -o program5a
praks@praks:~$ ./program5a
Producer sent 1
Consumer got 1
Producer sent 2
Consumer got 2
Producer sent 3
Consumer got 3
Consumer received 100000 integers
Time taken = 0.037467 sec
Data sent  = 400000 bytes
Speed      = 10.68 MB/s
praks@praks:~$ |
```

Explanation: The parent (producer) writes 100000 integers into the pipe and the child (consumer) reads them. A pipe blocks the producer when it is full and the consumer when it is empty, so they stay synchronized. 400000 bytes were transferred in 0.037467 sec, which is about 10.68 MB/s.

5(b). Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program5b.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    int fd[2];  
    pipe(fd);  
  
    if (fork() == 0)  
    {  
        dup2(fd[1], 1);  
        close(fd[0]);  
        close(fd[1]);  
        execlp("ls", "ls", "-l", NULL);  
        exit(1);  
    }  
  
    if (fork() == 0)  
    {  
        dup2(fd[0], 0);  
        close(fd[0]);  
        close(fd[1]);  
        execlp("grep", "grep", ".c", NULL);  
        exit(1);  
    }  
  
    close(fd[0]);  
    close(fd[1]);  
    wait(NULL);  
    wait(NULL);  
  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc program5b.c -o program5b
praks@praks:~$ ./program5b
-rw-r--r-- 1 praks praks 467 Sep 29 03:59 client.c
-rw-r--r-- 1 praks praks 26 Sep 29 04:02 copy.txt
-rw-r--r-- 1 praks praks 929 Sep 29 03:59 deadlock.c
-rw-r--r-- 1 praks praks 929 Sep 29 03:59 nodeadlock.c
-rw-r--r-- 1 praks praks 356 Sep 29 03:59 nozombie.c
-rw-r--r-- 1 praks praks 1162 Sep 29 03:59 program10b.c
-rw-r--r-- 1 praks praks 447 Sep 29 03:59 program11a.c
-rw-r--r-- 1 praks praks 581 Sep 29 03:59 program11b.c
-rw-r--r-- 1 praks praks 1548 Sep 29 03:59 program12a.c
-rw-r--r-- 1 praks praks 755 Sep 29 03:59 program1a.c
-rw-r--r-- 1 praks praks 476 Sep 29 03:59 program2a.c
-rw-r--r-- 1 praks praks 579 Sep 29 03:59 program3a.c
-rw-r--r-- 1 praks praks 822 Sep 29 03:59 program4a.c
-rw-r--r-- 1 praks praks 1080 Sep 29 03:59 program5a.c
-rw-r--r-- 1 praks praks 532 Sep 29 03:59 program5b.c
-rw-r--r-- 1 praks praks 545 Sep 29 03:59 program6b.c
-rw-r--r-- 1 praks praks 278 Sep 29 03:59 program7b.c
-rw-r--r-- 1 praks praks 909 Sep 29 04:01 program9a.c
-rw-r--r-- 1 praks praks 391 Sep 29 03:59 program9b.c
-rw-r--r-- 1 praks praks 661 Sep 29 03:59 server.c
-rw-r--r-- 1 praks praks 7943 Sep 29 04:02 trace.txt
-rw-r--r-- 1 praks praks 314 Sep 29 03:59 zombie.c
praks@praks:~$ |
```

Explanation: pipe() gives two descriptors. The first child does dup2(fd[1], 1) so the output of ls -l goes into the pipe, and the second child does dup2(fd[0], 0) so grep reads from the pipe. exec replaces each child with ls and grep.

6(a). Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 server.c  
#include <stdio.h>  
#include <fcntl.h>  
#include <ctype.h>  
#include <unistd.h>  
#include <sys/stat.h>  
  
int main()  
{  
    char buf[100];  
    int fd, n, i;  
  
    mkfifo("req_fifo", 0666);  
    mkfifo("res_fifo", 0666);  
    printf("Server started\n");  
  
    while (1)  
    {  
        fd = open("req_fifo", O_RDONLY);  
        n = read(fd, buf, sizeof(buf) - 1);  
        close(fd);  
        if (n <= 0)  
            continue;  
        buf[n] = '\0';  
        printf("Server received : %s\n", buf);  
  
        for (i = 0; i < n; i++)  
            buf[i] = toupper(buf[i]);  
  
        fd = open("res_fifo", O_WRONLY);  
        write(fd, buf, n);  
        close(fd);  
    }  
    return 0;  
}
```

```
praks@praks: ~  
GNU nano 8.7.1 client.c  
#include <stdio.h>  
#include <fcntl.h>  
#include <string.h>  
#include <unistd.h>  
  
int main(int argc, char *argv[])  
{  
    char msg[100], buf[100];  
    int fd, n;  
  
    sprintf(msg, "hello from %s", argv[1]);  
  
    fd = open("req_fifo", O_WRONLY);  
    write(fd, msg, strlen(msg));  
    close(fd);  
  
    fd = open("res_fifo", O_RDONLY);  
    n = read(fd, buf, sizeof(buf) - 1);  
    buf[n] = '\0';  
    close(fd);  
  
    printf("%s got reply : %s\n", argv[1], buf);  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc server.c -o server  
praks@praks:~$ gcc client.c -o client  
praks@praks:~$ ./server &  
[1] 394  
Server started  
praks@praks:~$ ./client client1  
Server received : hello from client1  
client1 got reply : HELLO FROM CLIENT1  
praks@praks:~$ ./client client2 & ./client client3  
[2] 396  
Server received : hello from client3  
client3 got reply : HELLO FROM CLIENT3  
Server received : hello from client2  
praks@praks:~$ client2 got reply : HELLO FROM CLIENT2  
[2]+ Done ./client client2  
praks@praks:~$ kill %1  
[1]+ Terminated ./server  
praks@praks:~$ |
```

Explanation: The server reads a message from req_fifo, converts it to upper case and writes it to res_fifo. A FIFO is only a byte stream without message boundaries and all clients share the same res_fifo, so when client2 and client3 run together their requests are handled one at a time, and a reply may reach the wrong client or two messages may get merged. To avoid this each client should get its own reply FIFO (for example named with its PID).

6(b). Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program6b.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <signal.h>  
#include <unistd.h>  
  
void handler(int sig)  
{  
    if (sig == SIGINT)  
        printf("Caught SIGINT\n");  
    else if (sig == SIGUSR1)  
        printf("Caught SIGUSR1\n");  
    else if (sig == SIGTERM)  
    {  
        printf("Caught SIGTERM, exiting\n");  
        exit(0);  
    }  
}  
  
int main()  
{  
    signal(SIGINT, handler);  
    signal(SIGTERM, handler);  
    signal(SIGUSR1, handler);  
  
    printf("PID = %d, waiting for signals...\n", getpid());  
  
    while (1)  
        pause();  
  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc program6b.c -o program6b  
praks@praks:~$ ./program6b &  
[1] 403  
praks@praks:~$ PID = 403, waiting for signals...  
praks@praks:~$ kill -SIGUSR1 $!  
praks@praks:~$ Caught SIGUSR1  
praks@praks:~$ kill -SIGINT $!  
praks@praks:~$ Caught SIGINT  
praks@praks:~$ kill -SIGTERM $!  
praks@praks:~$ Caught SIGTERM, exiting  
[1]+  Done                  ./program6b  
praks@praks:~$ |
```

Explanation: A signal is an asynchronous event sent to a process (by kill or the keyboard). signal() registers a handler function. When the signal arrives, the kernel interrupts the normal flow of the program, runs the handler and then the program continues. Here SIGUSR1 and SIGINT were caught and printed, and SIGTERM was caught and the program exited.

7(a). Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program7.c  
#include <stdio.h>  
#include <stdlib.h>  
  
int global = 100;  
static int s = 50;  
  
int main()  
{  
    int stack = 10;  
    int *heap = (int *)malloc(sizeof(int));  
  
    printf("Code Segment : %p\n", (void *)main);  
    printf("Global Segment : %p\n", (void *)&global);  
    printf("Static Segment : %p\n", (void *)&s);  
    printf("Heap Segment : %p\n", (void *)heap);  
    printf("Stack Segment : %p\n", (void *)&stack);  
  
    free(heap);  
  
    return 0;  
}
```

Output:

```
praks@praks: ~  
praks@praks:~$ nano program7.c  
praks@praks:~$ gcc program7.c -o program7  
praks@praks:~$ ./program7  
Code Segment : 0x5e33542d81a9  
Global Segment : 0x5e33542db010  
Static Segment : 0x5e33542db014  
Heap Segment : 0x5e3391aa5010  
Stack Segment : 0x7ffeddbf4e8c  
praks@praks:~$ |
```

Explanation: Code segment: machine instructions of the program (main). Global and static segment: global and static variables, which live for the whole run (their addresses are next to each other). Heap: memory from malloc(), grows upwards. Stack: local variables and function calls, grows downwards. The output shows code, global and static at nearby low addresses, the heap a little higher, and the stack at a very high address (0x7ffe...).

7(b). Make use of the `/proc/<pid>/maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program7b.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
  
int global = 100;  
  
int main()  
{  
    int stack = 10;  
    int *heap = (int *)malloc(1000);  
  
    printf("PID = %d\n", getpid());  
    printf("Sleeping for 30 seconds...\n");  
    sleep(30);  
  
    free(heap);  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc program7b.c -o program7b  
praks@praks:~$ ./program7b &  
[1] 409  
praks@praks:~$ PID = 409  
Sleeping for 30 seconds...  
praks@praks:~$ awk '{print $1, $2, $6}' /proc/$!/maps  
55ceb7bb4000-55ceb7bb5000 r--p /home/praks/program7b  
55ceb7bb5000-55ceb7bb6000 r-xp /home/praks/program7b  
55ceb7bb6000-55ceb7bb7000 r--p /home/praks/program7b  
55ceb7bb7000-55ceb7bb8000 r--p /home/praks/program7b  
55ceb7bb8000-55ceb7bb9000 rw-p /home/praks/program7b  
55ced2bda000-55ced2bfb000 rw-p [heap]  
7f46cea00000-7f46cea28000 r--p /usr/lib/x86_64-linux-gnu/libc.so.6  
7f46cea28000-7f46cebb1000 r-xp /usr/lib/x86_64-linux-gnu/libc.so.6  
7f46cebb1000-7f46cec00000 r--p /usr/lib/x86_64-linux-gnu/libc.so.6  
7f46cec00000-7f46cec04000 r--p /usr/lib/x86_64-linux-gnu/libc.so.6  
7f46cec04000-7f46cec06000 rw-p /usr/lib/x86_64-linux-gnu/libc.so.6  
7f46cec06000-7f46cec13000 rw-p  
7f46ced77000-7f46ced7a000 rw-p  
7f46ced87000-7f46ced89000 rw-p  
7f46ced89000-7f46ced8d000 r--p [vvar]  
7f46ced8d000-7f46ced8f000 r--p [vvar_vclock]  
7f46ced8f000-7f46ced91000 r-xp [vdso]  
7f46ced91000-7f46ced92000 r--p /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7f46ced92000-7f46cedbd000 r-xp /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7f46cedbd000-7f46cedc7000 r--p /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7f46cedc7000-7f46cedc9000 r--p /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7f46cedc9000-7f46cedcb000 rw-p /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7ffeac45f000-7ffeac480000 rw-p [stack]  
ffffffffffff600000-ffffffffffff601000 --xp [vsyscall]  
praks@praks:~$ |
```

```

praks@praks:~$ pmap $!
409:  ./program7b
000055ceb7bb4000      4K r---- program7b
000055ceb7bb5000      4K r-x-- program7b
000055ceb7bb6000      4K r---- program7b
000055ceb7bb7000      4K r---- program7b
000055ceb7bb8000      4K rw--- program7b
000055ced2bda000    132K rw--- [ anon ]
00007f46cea00000    160K r---- libc.so.6
00007f46cea28000   1572K r-x-- libc.so.6
00007f46cebb1000    316K r---- libc.so.6
00007f46cec00000     16K r---- libc.so.6
00007f46cec04000      8K rw--- libc.so.6
00007f46cec06000     52K rw--- [ anon ]
00007f46ced77000     12K rw--- [ anon ]
00007f46ced87000      8K rw--- [ anon ]
00007f46ced89000     16K r---- [ anon ]
00007f46ced8d000      8K r---- [ anon ]
00007f46ced8f000      8K r-x-- [ anon ]
00007f46ced91000      4K r---- ld-linux-x86-64.so.2
00007f46ced92000    172K r-x-- ld-linux-x86-64.so.2
00007f46cedbd000     40K r---- ld-linux-x86-64.so.2
00007f46cedc7000      8K r---- ld-linux-x86-64.so.2
00007f46cedc9000      8K rw--- ld-linux-x86-64.so.2
00007ffeac45f000    132K rw--- [ stack ]
fffffffffff60000      4K --x-- [ anon ]
total                2696K
praks@praks:~$ |

```

Explanation: maps lists every virtual memory region of the process: program code (r-xp), read-only data, global data (rw-p), [heap], shared libraries (libc.so.6, ld-linux), [vdso] and [stack]. Each line gives the address range, permissions (r, w, x, p = private) and the mapped file. pmap shows the same regions with their sizes. These are virtual addresses: every process has its own address space and the kernel maps it to physical memory pages when needed.

8(a). Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Program:

```
praks@praks: ~
GNU nano 8.7.1 program8.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    a = (int *)malloc(5 * sizeof(int));

    b = (int *)calloc(5, sizeof(int));

    a = (int *)realloc(a, 10 * sizeof(int));

    free(a);
    free(b);

    return 0;
}
```

Output:

```
praks@praks:~$ gcc program8.c -o program8
praks@praks:~$ valgrind --leak-check=full ./program8
==2048== Memcheck, a memory error detector
==2048== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2048== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2048== Command: ./program8
==2048==
==2048==
==2048== HEAP SUMMARY:
==2048==     in use at exit: 0 bytes in 0 blocks
==2048==   total heap usage: 3 allocs, 3 frees, 80 bytes allocated
==2048==
==2048== All heap blocks were freed -- no leaks are possible
==2048==
==2048== For lists of detected and suppressed errors, rerun with: -s
==2048== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
praks@praks:~$ |
```

Explanation: malloc() allocates 5 ints (not initialized), calloc() allocates 5 ints set to zero, realloc() resizes the first block to 10 ints, and free() releases both. Valgrind shows 3 allocs and 3 frees, 0 bytes in use at exit, so there is no memory leak.

8(b). Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 cow.c  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    int x = 100;  
  
    if (fork() == 0)  
    {  
        printf("Child Before = %d\n", x);  
  
        x = 500;  
  
        printf("Child After = %d\n", x);  
    }  
    else  
    {  
        sleep(2);  
  
        printf("Parent = %d\n", x);  
    }  
  
    return 0;  
}
```

Output:

```
praks@praks:~$ sudo dpkg --configure -a  
praks@praks:~$ nano cow.c  
praks@praks:~$ gcc cow.c -o cow  
praks@praks:~$ ./cow  
Child Before = 100  
Child After = 500  
Parent = 100  
praks@praks:~$ |
```

Explanation: After fork() the parent and child share the same physical pages. When the child writes x = 500, the kernel makes a private copy of that page for the child, so the parent still prints 100. No extra memory is used until a write happens.

9(a). Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program9a.c  
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
#include <time.h>  
  
int main()  
{  
    char buf[128];  
    int in, out, n;  
    FILE *fin, *fout;  
    clock_t t1, t2;  
  
    in = open("big.bin", O_RDONLY);  
    out = open("copy1.bin", O_WRONLY | O_CREAT | O_TRUNC, 0644);  
    printf("File size = %ld bytes\n", lseek(in, 0, SEEK_END));  
    lseek(in, 0, SEEK_SET);  
  
    t1 = clock();  
    while ((n = read(in, buf, sizeof(buf))) > 0)  
        write(out, buf, n);  
    t2 = clock();  
    close(in);  
    close(out);  
    printf("System calls : %f sec\n", (double)(t2 - t1) / CLOCKS_PER_SEC);  
  
    fin = fopen("big.bin", "rb");  
    fout = fopen("copy2.bin", "wb");  
  
    t1 = clock();  
    while ((n = fread(buf, 1, sizeof(buf), fin)) > 0)  
        fwrite(buf, 1, n, fout);  
    t2 = clock();  
    fclose(fin);  
    fclose(fout);  
    printf("Library funcs : %f sec\n", (double)(t2 - t1) / CLOCKS_PER_SEC);  
  
    return 0;  
}
```

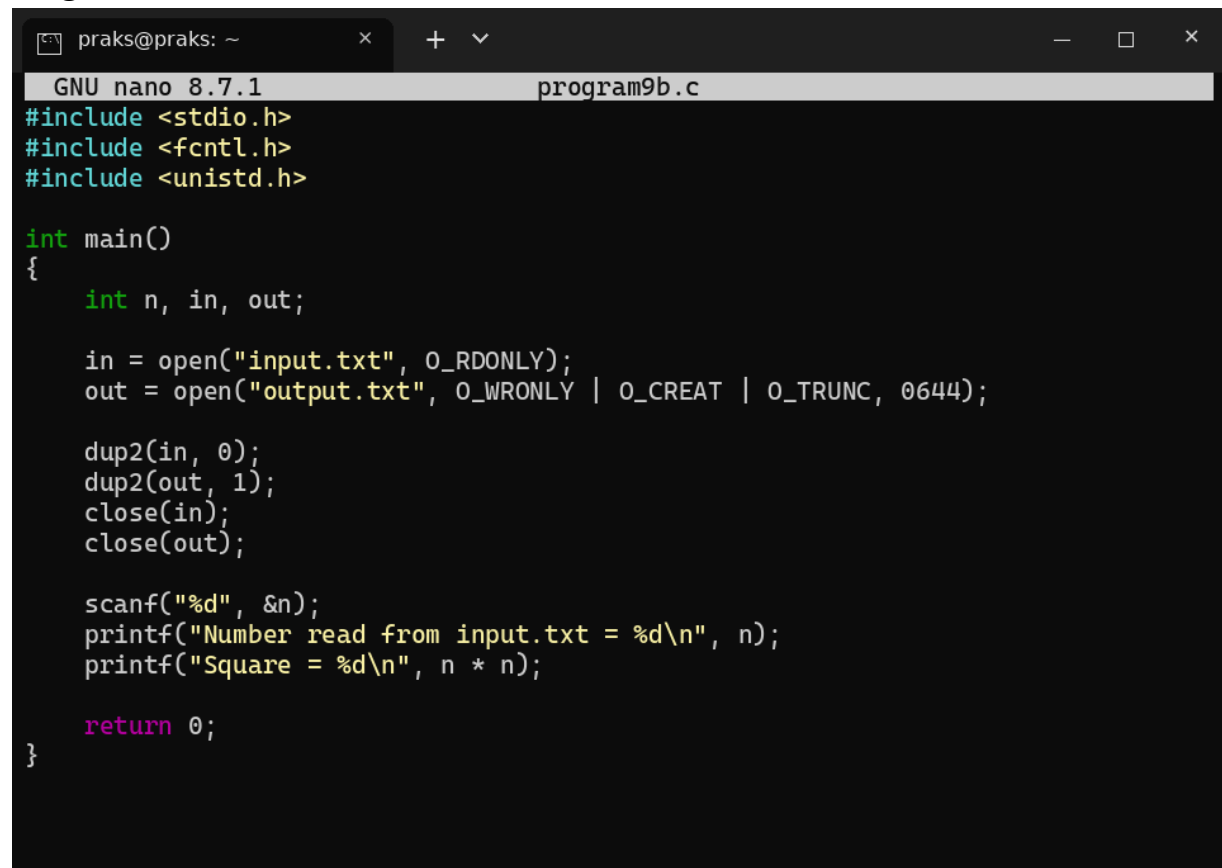
Output:

```
praks@praks:~$ dd if=/dev/urandom of=big.bin bs=1M count=20  
20+0 records in  
20+0 records out  
20971520 bytes (21 MB, 20 MiB) copied, 0.0707114 s, 297 MB/s  
praks@praks:~$ gcc program9a.c -o program9a  
praks@praks:~$ ./program9a  
File size = 20971520 bytes  
System calls : 0.080020 sec  
Library funcs : 0.022062 sec  
praks@praks:~$ |
```

Explanation: read() and write() are direct system calls, so with a small buffer (128 bytes) a large number of system calls is made (0.080020 sec). fread() and fwrite() collect data in an internal buffer, so far fewer system calls are needed and the library version is faster (0.022062 sec). With large buffers both are almost equal.

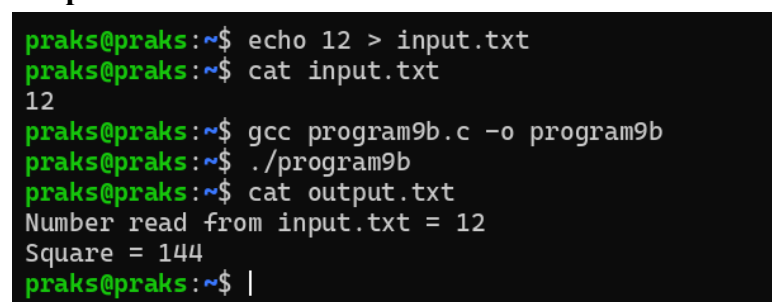
9(b). Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

Program:



```
praks@praks: ~  
GNU nano 8.7.1 program9b.c  
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
  
int main()  
{  
    int n, in, out;  
  
    in = open("input.txt", O_RDONLY);  
    out = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);  
  
    dup2(in, 0);  
    dup2(out, 1);  
    close(in);  
    close(out);  
  
    scanf("%d", &n);  
    printf("Number read from input.txt = %d\n", n);  
    printf("Square = %d\n", n * n);  
  
    return 0;  
}
```

Output:



```
praks@praks:~$ echo 12 > input.txt  
praks@praks:~$ cat input.txt  
12  
praks@praks:~$ gcc program9b.c -o program9b  
praks@praks:~$ ./program9b  
praks@praks:~$ cat output.txt  
Number read from input.txt = 12  
Square = 144  
praks@praks:~$ |
```

Explanation: dup2(in, 0) makes descriptor 0 (stdin) refer to input.txt and dup2(out, 1) makes descriptor 1 (stdout) refer to output.txt. So scanf() reads from the file and printf() writes to the file, and nothing is printed on the screen. A shell does the same for ./program < input.txt > output.txt: it forks, calls dup2() in the child and then runs the program with exec.

10(a). Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Commands and Output:

```
praks@praks:~$ echo "hello inode" > file1.txt
praks@praks:~$ ls -li file1.txt
582142 file1.txt
praks@praks:~$ stat file1.txt
  File: file1.txt
  Size: 12          Blocks: 8          IO Block: 4096   regular file
Device: 254,0    Inode: 582142     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1001/   praks)   Gid: ( 1002/   praks)
Access: 2026-09-29 04:03:40.049412958 +0000
Modify: 2026-09-29 04:03:40.049412958 +0000
Change: 2026-09-29 04:03:40.049412958 +0000
 Birth: 2026-09-29 04:03:40.049412958 +0000
praks@praks:~$ |
```

```
praks@praks:~$ ln file1.txt hardlink.txt
praks@praks:~$ ln -s file1.txt softlink.txt
praks@praks:~$ ls -li file1.txt hardlink.txt softlink.txt
582142 -rw-r--r-- 2 praks praks 12 Sep 29 04:03 file1.txt
582142 -rw-r--r-- 2 praks praks 12 Sep 29 04:03 hardlink.txt
582143 lrwxrwxrwx 1 praks praks  9 Sep 29 04:03 softlink.txt -> file1.txt
praks@praks:~$ find . -inum 582142
./hardlink.txt
./file1.txt
praks@praks:~$ |
```

Explanation: A file name is only a directory entry that points to an inode; the inode stores the size, owner, permissions, timestamps and data block pointers. A hard link (`hardlink.txt`) is another name for the same inode: the inode number is the same, the link count becomes 2 and no new inode is allocated. A symbolic link is a separate file with its own inode that stores the path of the target, so it uses a new inode.

10(b). Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program10b.c  
#include <stdio.h>  
#include <fcntl.h>  
#include <string.h>  
#include <unistd.h>  
#include <sys/mman.h>  
#include <sys/stat.h>  
#include <time.h>  
  
int main()  
{  
    int fd = open("sample.txt", O_RDWR);  
    struct stat st;  
    fstat(fd, &st);  
  
    char *p = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);  
  
    printf("Read using mmap : %.20s\n", p);  
    memcpy(p, "HELLO", 5);  
    msync(p, st.st_size, MS_SYNC);  
    munmap(p, st.st_size);  
    close(fd);  
  
    /* speed comparison on big.bin */  
    struct stat bs;  
    long sum = 0, i;  
    char buf[4096];  
    int n;  
    clock_t t1, t2;  
  
    fd = open("big.bin", O_RDONLY);  
    fstat(fd, &bs);  
  
    t1 = clock();  
    while ((n = read(fd, buf, sizeof(buf))) > 0)  
        for (i = 0; i < n; i++)  
            sum += buf[i];  
    t2 = clock();  
    printf("read() : %f sec\n", (double)(t2 - t1) / CLOCKS_PER_SEC);  
  
    sum = 0;  
    lseek(fd, 0, SEEK_SET);  
    char *m = mmap(NULL, bs.st_size, PROT_READ, MAP_PRIVATE, fd, 0);  
  
    t1 = clock();  
    for (i = 0; i < bs.st_size; i++)  
        sum += m[i];  
    t2 = clock();  
    printf("mmap() : %f sec\n", (double)(t2 - t1) / CLOCKS_PER_SEC);  
  
    return 0;  
}
```

Output:

```
praks@praks:~$ cat sample.txt
Welcome to OSSP practical
praks@praks:~$ gcc program10b.c -o program10b
praks@praks:~$ ./program10b
Read using mmap : Welcome to OSSP prac
read() : 0.043910 sec
mmap() : 0.046166 sec
praks@praks:~$ cat sample.txt
HELLOme to OSSP practical
praks@praks:~$ |
```

Explanation: mmap() maps the file into the address space, so the contents are used like a normal array and are changed with memcpy() (the first 5 letters of sample.txt became HELLO) without read() or write() calls. For the 20 MB file read() took 0.043910 sec and mmap() took 0.046166 sec, so speed is similar here; mmap avoids the extra copy and repeated system calls and is best for large files and random access, but it is harder to code (msync, munmap, fixed file size, page faults).

11(a). Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Program:

```
praks@praks: ~
GNU nano 8.7.1 program11a.c
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < 1000000; i++)
        counter++;
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Expected = 2000000\n");
    printf("Actual    = %d\n", counter);
    return 0;
}
```

Output:

```
praks@praks:~$ gcc program11a.c -o program11a -pthread
praks@praks:~$ ./program11a
Expected = 2000000
Actual   = 1000000
praks@praks:~$ ./program11a
Expected = 2000000
Actual   = 2000000
praks@praks:~$ ./program11a
Expected = 2000000
Actual   = 2000000
praks@praks:~$ |
```

Explanation: Both threads run `counter++` on the same variable. `counter++` is really load, add and store, so if a thread is switched out in between, an update is lost. The actual value can therefore be less than the expected 2000000 and may change from run to run. This is a race condition.

11(b). Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

Program:

```
praks@praks: ~
GNU nano 8.7.1 program11b.c
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg)
{
    for (int i = 0; i < 1000000; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Expected = 2000000\n");
    printf("Actual   = %d\n", counter);
    return 0;
}
```

Output:

```
praks@praks:~$ gcc program11b.c -o program11b -pthread
praks@praks:~$ ./program11b
Expected = 2000000
Actual   = 2000000
praks@praks:~$ ./program11b
Expected = 2000000
Actual   = 2000000
praks@praks:~$ |
```

Explanation: The mutex allows only one thread at a time to update counter, so no update is lost and the result is always exactly 2000000. Synchronization makes the program correct, but it is slower because threads have to wait for the lock.

12(a). Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 program12a.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <pthread.h>  
#include <semaphore.h>  
#include <sys/time.h>  
  
int *buffer, size, items, in = 0, out = 0;  
sem_t empty, full;  
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;  
  
void *producer(void *arg)  
{  
    for (int i = 1; i <= items; i++)  
    {  
        sem_wait(&empty);  
        pthread_mutex_lock(&mutex);  
        buffer[in] = i;  
        in = (in + 1) % size;  
        if (items <= 10) printf("Produced %d\n", i);  
        pthread_mutex_unlock(&mutex);  
        sem_post(&full);  
    }  
    return NULL;  
}  
  
void *consumer(void *arg)  
{  
    for (int i = 1; i <= items; i++)  
    {  
        sem_wait(&full);  
        pthread_mutex_lock(&mutex);  
        int x = buffer[out];  
        out = (out + 1) % size;  
        if (items <= 10) printf("Consumed %d\n", x);  
        pthread_mutex_unlock(&mutex);  
        sem_post(&empty);  
    }  
    return NULL;  
}  
  
int main(int argc, char *argv[])  
{  
    pthread_t p, c;  
    struct timeval t1, t2;  
  
    size = atoi(argv[1]);  
    items = atoi(argv[2]);  
    buffer = malloc(size * sizeof(int));  
  
    sem_init(&empty, 0, size);  
    sem_init(&full, 0, 0);  
  
    gettimeofday(&t1, NULL);  
    pthread_create(&p, NULL, producer, NULL);  
    pthread_create(&c, NULL, consumer, NULL);  
    pthread_join(p, NULL);  
    pthread_join(c, NULL);  
    gettimeofday(&t2, NULL);  
  
    double t = (t2.tv_sec - t1.tv_sec) + (t2.tv_usec - t1.tv_usec) / 1e6;  
    printf("Buffer = %d, Items = %d, Time = %f sec, Throughput = %.0f items/sec\n",  
           size, items, t, items / t);  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc program12a.c -o program12a -pthread
praks@praks:~$ ./program12a 3 10
Produced 1
Produced 2
Produced 3
    Consumed 1
Produced 4
    Consumed 2
Produced 5
    Consumed 3
Produced 6
    Consumed 4
Produced 7
    Consumed 5
Produced 8
    Consumed 6
Produced 9
    Consumed 7
Produced 10
    Consumed 8
    Consumed 9
    Consumed 10
Buffer = 3, Items = 10, Time = 0.000318 sec, Throughput = 31447 items/sec
praks@praks:~$ |
```

```
praks@praks:~$ ./program12a 1 100000
Buffer = 1, Items = 100000, Time = 0.153430 sec, Throughput = 651763 items/sec
praks@praks:~$ ./program12a 10 100000
Buffer = 10, Items = 100000, Time = 0.056203 sec, Throughput = 1779264 items/sec
praks@praks:~$ ./program12a 100 100000
Buffer = 100, Items = 100000, Time = 0.037773 sec, Throughput = 2647394 items/sec
c
praks@praks:~$ |
```

Explanation: empty counts free slots (starts at the buffer size), full counts filled slots (starts at 0) and the mutex protects the buffer. The producer waits when the buffer is full and the consumer waits when it is empty, so every item is consumed exactly once and in order. Throughput increases with the buffer size: 651,763 items/sec for buffer 1, 1,779,264 for buffer 10 and 2,647,394 for buffer 100, because the threads block and switch less often.

12(b). Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Program:

```
praks@praks: ~  
GNU nano 8.7.1 deadlock.c  
#include <stdio.h>  
#include <pthread.h>  
#include <unistd.h>  
  
pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;  
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;  
  
void *thread1(void *arg)  
{  
    pthread_mutex_lock(&A);  
    printf("Thread 1 locked A\n");  
    sleep(1);  
    printf("Thread 1 waiting for B\n");  
    pthread_mutex_lock(&B);  
    printf("Thread 1 locked B\n");  
    pthread_mutex_unlock(&B);  
    pthread_mutex_unlock(&A);  
    return NULL;  
}  
  
void *thread2(void *arg)  
{  
    pthread_mutex_lock(&B);  
    printf("Thread 2 locked B\n");  
    sleep(1);  
    printf("Thread 2 waiting for A\n");  
    pthread_mutex_lock(&A);  
    printf("Thread 2 locked A\n");  
    pthread_mutex_unlock(&A);  
    pthread_mutex_unlock(&B);  
    return NULL;  
}  
  
int main()  
{  
    pthread_t t1, t2;  
    pthread_create(&t1, NULL, thread1, NULL);  
    pthread_create(&t2, NULL, thread2, NULL);  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
    return 0;  
}
```

```
praks@praks: ~  
GNU nano 8.7.1 nodeadlock.c  
#include <stdio.h>  
#include <pthread.h>  
#include <unistd.h>  
  
pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;  
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;  
  
void *thread1(void *arg)  
{  
    pthread_mutex_lock(&A);  
    printf("Thread 1 locked A\n");  
    sleep(1);  
    printf("Thread 1 waiting for B\n");  
    pthread_mutex_lock(&B);  
    printf("Thread 1 locked B\n");  
    pthread_mutex_unlock(&B);  
    pthread_mutex_unlock(&A);  
    return NULL;  
}  
  
void *thread2(void *arg)  
{  
    pthread_mutex_lock(&A);  
    printf("Thread 2 locked A\n");  
    sleep(1);  
    printf("Thread 2 waiting for B\n");  
    pthread_mutex_lock(&B);  
    printf("Thread 2 locked B\n");  
    pthread_mutex_unlock(&B);  
    pthread_mutex_unlock(&A);  
    return NULL;  
}  
  
int main()  
{  
    pthread_t t1, t2;  
    pthread_create(&t1, NULL, thread1, NULL);  
    pthread_create(&t2, NULL, thread2, NULL);  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
    return 0;  
}
```

Output:

```
praks@praks:~$ gcc deadlock.c -o deadlock -pthread
praks@praks:~$ gcc nodeadlock.c -o nodeadlock -pthread
praks@praks:~$ ./deadlock
Thread 1 locked A
Thread 2 locked B
Thread 2 waiting for A
Thread 1 waiting for B
^C
praks@praks:~$ ./nodeadlock
Thread 1 locked A
Thread 1 waiting for B
Thread 1 locked B
Thread 2 locked A
Thread 2 waiting for B
Thread 2 locked B
praks@praks:~$ |
```

Explanation: In `deadlock.c` Thread 1 holds A and waits for B while Thread 2 holds B and waits for A, so both wait forever (stopped with Ctrl+C). Four necessary conditions: (1) mutual exclusion - a mutex can be held by only one thread, (2) hold and wait - each thread holds one lock while waiting for the other, (3) no preemption - a lock cannot be taken away, (4) circular wait - Thread 1 waits for Thread 2 and Thread 2 waits for Thread 1. Prevention: in `nodeadlock.c` both threads lock in the same order (A then B), which removes circular wait, so both threads finish.